

Java EE

JSP – JAVA SERVER PAGES

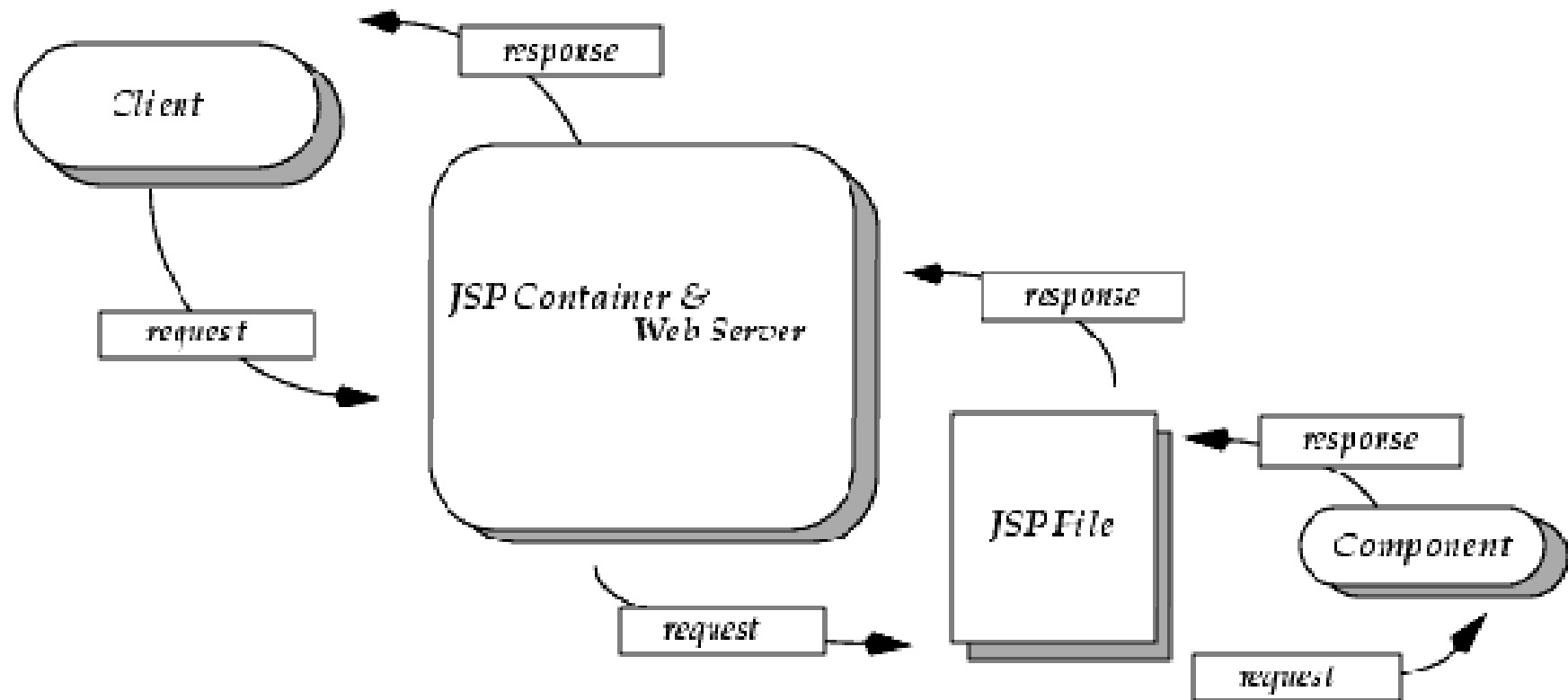
Contenu

- Introduction
- Fonctionnement
- Syntaxe
 - Balises simples
 - Directives
 - Actions standards
 - Expression language EL

Introduction

- Éviter le code java sur les vues
- Une page JSP contient :
 - du contenu statique
 - du code JSP : produit dynamiquement du contenu

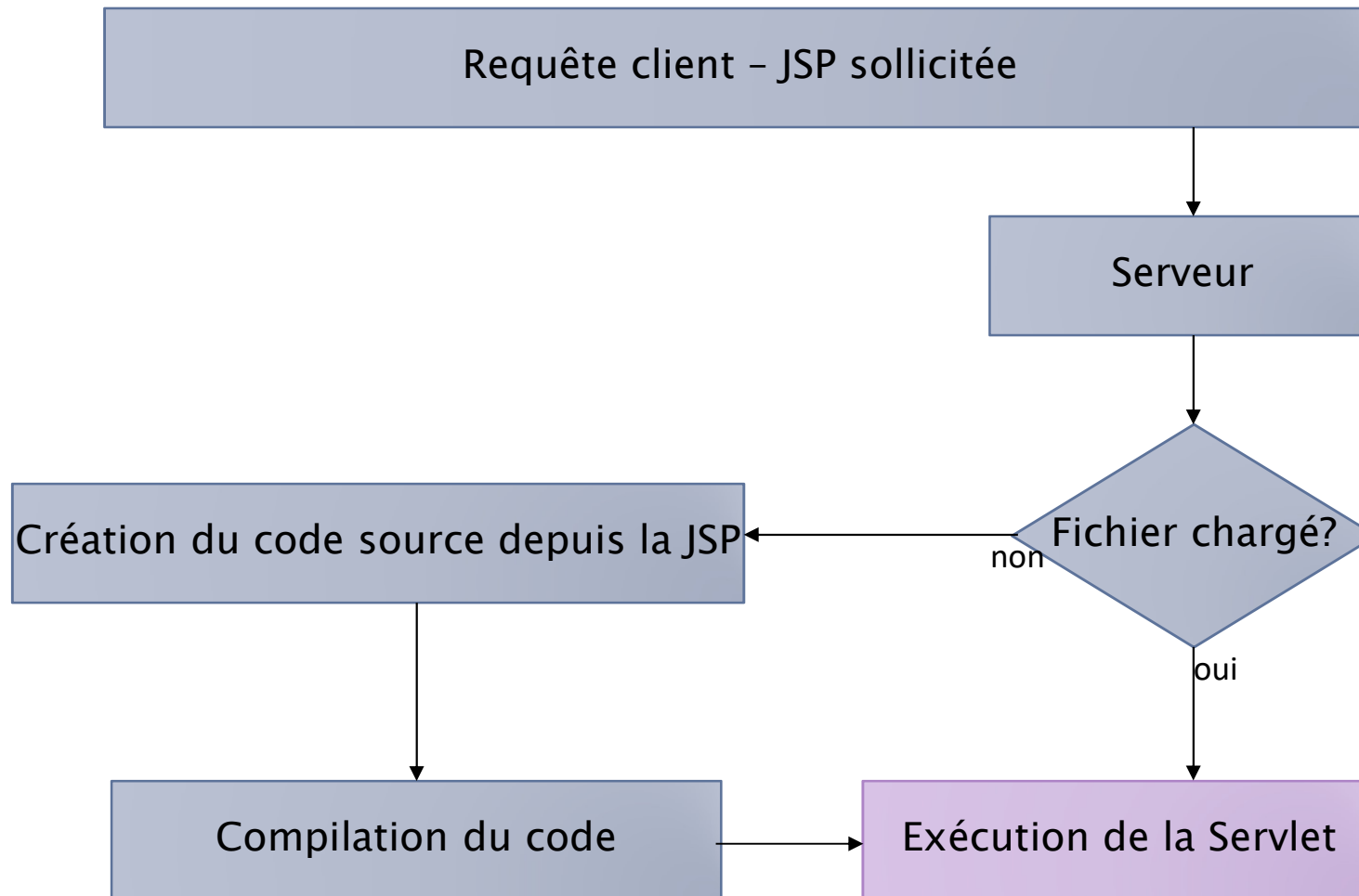
Principe de chargement



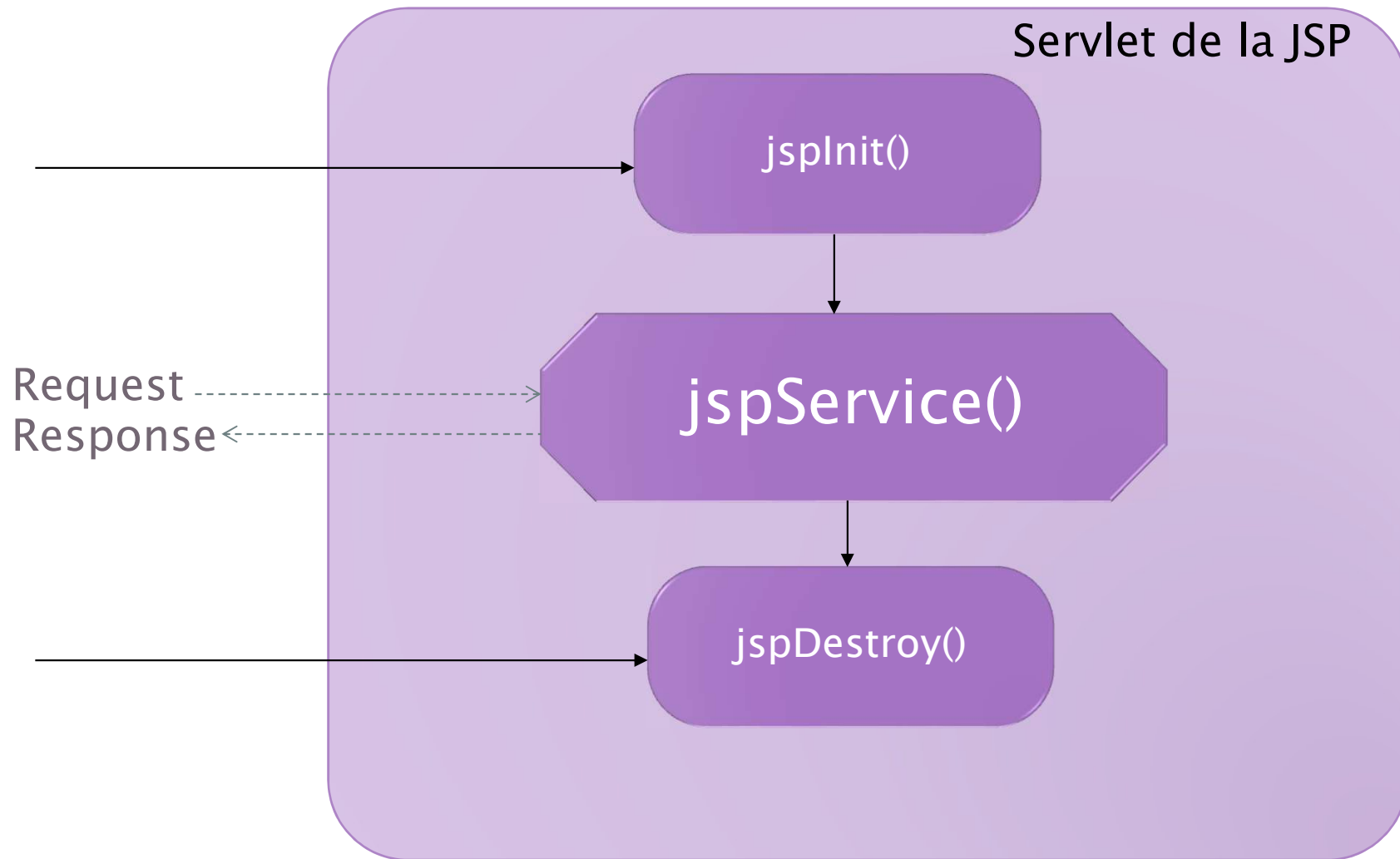
Fonctionnement

- Traduction : JSP ➔ en servlet
- Fonctionnement :
 - Étape 1 :
 - Requête sur une JSP
 - Étape 2 :
 - une servlet compare les dates de la JSP et de sa servlet
 - si la servlet de la JSP est plus ancienne que la page :
 - traduction de la JSP en servlet
 - compilation de la servlet
 - Étape 3 :
 - si la servlet de la page n'a pas encore été chargée
 - chargement de la servlet
 - instanciation
 - initialise la servlet avec la méthode jsplnit
 - Étape 4 :
 - invocation de la méthode jspService
 - Étape 5 :
 - Appel de jspDestroy lors du déchargement de la servlet

Fonctionnement



Cycle de vie d'une JSP

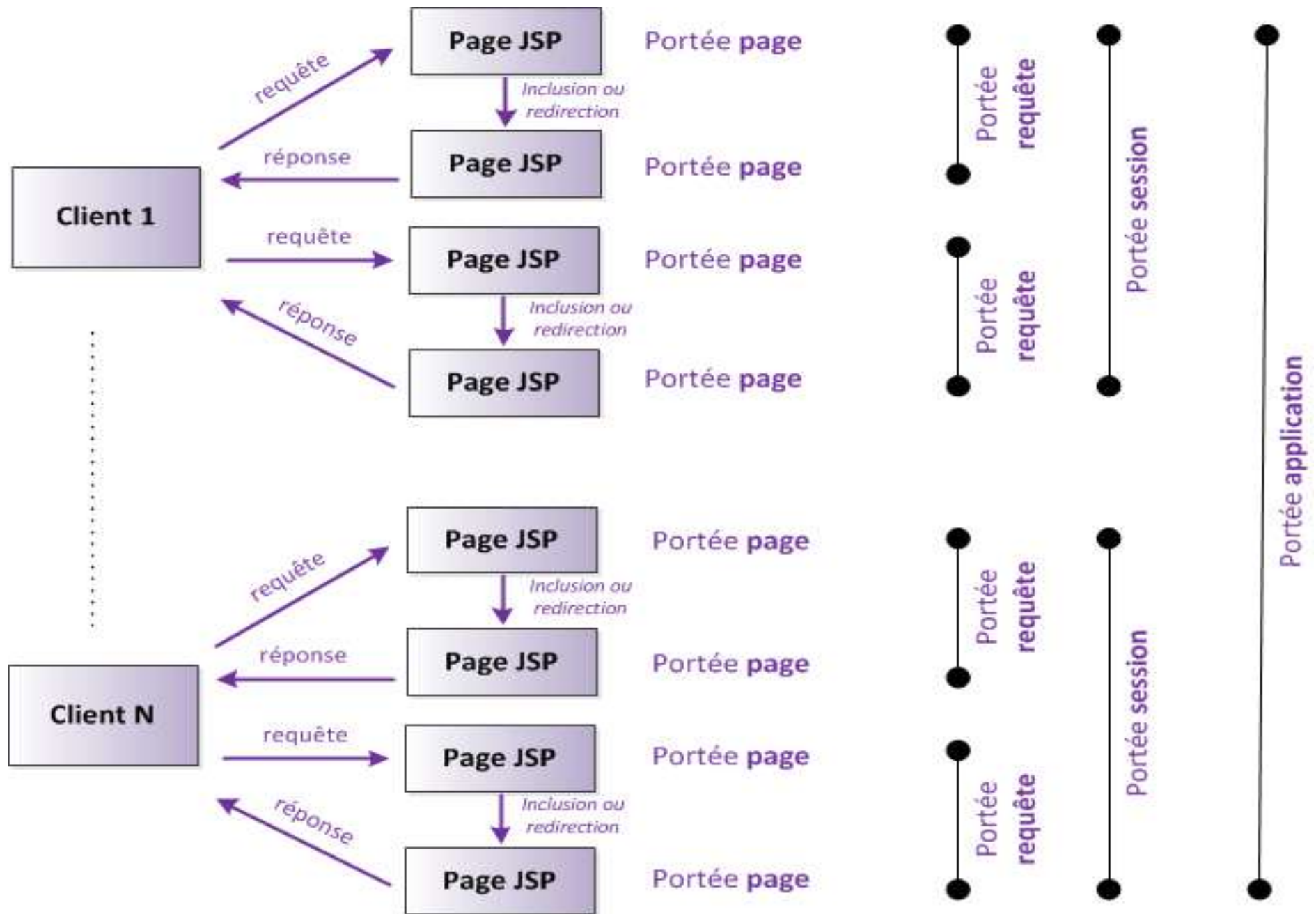


Éléments prédéfinis

- **request** – HttpServletRequest (paramètres de la requête, entête HTTP, cookies, ...), correspondant à la requête
- **response** – HttpServletResponse, correspondant à la réponse renvoyée
- **out** – PrintWriter, le flot de sortie de la réponse
- **session** – HttpSession, permet de gérer une session
- **application** - ServletContext partagée par toutes les servlets en cours ; contient, entre autres, la méthode log() pour écrire dans le fichier log
- **config** - ServletConfig
- **pageContext** - PageContext dans la JSP comme unique point d'accès, permet la gestion du contexte et des attributs de la JSP
- **page** – variable synonyme de cet objet (**this**)

Porté des objets

- Il existe au total quatre portées différentes dans une application :
 - **page** : les objets dans cette portée sont uniquement accessibles dans la page JSP en question ;
 - **requête** : les objets dans cette portée sont uniquement accessibles durant l'existence de la requête en cours ;
 - **session** : les objets dans cette portée sont accessibles durant l'existence de la session en cours ;
 - **application** : les objets dans cette portée sont accessibles durant toute l'existence de l'application



Exercice

- Ajouter le code nécessaire au projet test pour garder trace dans un fichier log des paramètres passés à la servlet depuis la JSP (seulement dans le cas où des paramètres sont utilisés).

Initialisation et finalisation : paramétrage

- Initialisation lors du chargement par le serveur d'application :
 - `public void jspInit()`
- Finalisation lors du déchargement par le serveur d'application :
 - `public void jspDestroy()`
- Paramétrage à placer dans une balise de déclaration

Balises simples

- Balises de commentaire
 - `<%-- et --%>`
- Balise de déclaration
 - Syntaxe :
 - `<%! Type nomVar = initialisation; %>`
 - Exemple :
 - `<%! String chaine = "azerty"; %>`
 - Exemple de bloc de déclaration :
 - `<%! String test = null;`
 - `public boolean fct() { if (test==null) return true; else return false; }`
 - `%>`
 - Traduction :
 - Déclarations : placées dans la classe au moment de la traduction de la JSP en Servlet
 - variables = attributs

Autres balises simples

- Balise de scriptlet :
 - Mélange de script et de servlet
 - Exemple :
 - `<%`
 - `String attributChaine = (String) request.getAttribute("attribut1Chaine");`
 - `out.println( attributChaine );`
 - `%>`
- Balise d'expression :
 - `<%= " Chaine à afficher" %>`
 - Correspond à : `<% out.println(" Chaine à afficher "); %>`

Directives

- Les directives JSP permettent :
 - d'importer un package ;
 - d'inclure d'autres pages JSP ;
 - d'inclure des bibliothèques de balises ;
 - de définir des propriétés et informations relatives à une page JSP.
- Comprises entre les balises `<%@` et `%>`
- Emplacement des directives :
 - La directive d'inclusion de page : peut être placée n'importe où
 - Les autres directives : doivent être **en tête de page JSP**.

3 directives

- **Directive page:**
 - `<%@ page directive %>`
- **Directive include:**
 - `<%@ include directive %>`
- **Directive taglib:**
 - `<%@ taglib directive %>`

Directive de page

- Principe :
 - Paramétrage de l'exécution de la JSP
- Syntaxe :
 - `<%@ page directive %>`
- Exemple :
 - `<%@ page isThreadSafe="false" %>`
 - `<%@ page contentType="text/plain; charset=UTF-8" %>`
 - `<%@ page import="java.io.*, java.util.*" %>`

Propriétés de la directive page

```
<%@ page
    language="..."
    extends="..."
    import="..."
    session="true | false"
    buffer="none | 8kb | sizekb"
    autoFlush="true | false"
    isThreadSafe="true | false"
    isELIgnored="true | false"
    info="..."
    errorPage="..."
    contentType="..."
    pageEncoding="..."
    isErrorPage="true | false"
%>
```

Exemple de directive de page

- `<%@ page errorPage="le_nom_de_la_page"%>`
 - spécifie la page vers laquelle le serveur d'application renvoie lorsqu'une exception non gérée est lancée par la JSP.
- `<%@ page isErrorPage="true|false"%>`
 - spécifie si la page à renvoyer est une page d'erreur et lui autorise ainsi la transmission de l'exception pour un éventuel traitement.

Exercice

- Créer une page html ayant pour titre « Erreur !!! » qui affiche un texte rapportant la présence d'une erreur sur la JSP,
- Ajouter à Test.jsp la directive permettant l'affichage de cette page html si une exception se présente,
- Dans un bloc de scriptlet faire une opération de division par 0,
- Tester le projet.

Directive et action standard d'inclusion

- Principe :
 - Inclusion d'une page statique ou dynamique dans la JSP
- Syntaxe :
 - Inclusion à la traduction en servlet (par directive)
 - `<%@ include directive %>`
 - Inclusion à l'exécution (par action standard)
 - Sans paramètres :
 - `<jsp:include page="fichier " />`
 - Avec paramètres :
 - `<jsp:include page="fichier ">`
 - `<jsp:param name="nom " value="valeur "/>`
 - ...
 - `</jsp:include>`

Exercice

- Implémenter les fichiers suivants avec de nouvelles JSP (ne pas oublier la configuration dans web.xml) :
 - inclusion.jsp
 - `<html>`
 - `<head><title>JSP avec inclusion</title></head>`
 - `<body><%@include file="titre.txt" %>`
 - `<p><jsp:include page="texte.jsp" /></p>`
 - `<p><jsp:include page="parametre.jsp">`
 - `<jsp:param name="nom" value="Bob" />`
 - `</jsp:include>`
 - `</p>`
 - `</body>`
 - `</html>`
 - titre.txt
 - `<h1>Titre de la JSP, inclus a partir d'un fichier texte</h1>`
 - texte.jsp
 - `<% out.println("Texte genere par la JSP texte.jsp"); %>`
 - parametre.jsp
 - `<%`
 - `String nom = request.getParameter("nom");`
 - `out.println("Salut" + nom);`
 - `%>`

Action standard : forward

- Permet d'effectuer une redirection vers une autre page
- Action effectuée côté serveur, donc pas possible de rediriger vers une page extérieure à l'application
- Action limitée aux pages présentes dans le contexte de la servlet ou de la JSP utilisée
- Syntaxe :
 - `<jsp:forward page="/nouvelle_page.jsp" />`

Inclusion pour la délégation

- Principe :
 - Délégation à un autre composant web
- Syntaxe :
 - Sans paramètres :
 - `<jsp:forward page="fichier " />`
 - Avec paramètres :
 - `<jsp:forward page="fichier " >`
 - `<jsp:param name="nom " value="valeur "/>`
 - ...
 - `</jsp:forward>`

Exercice

- Implémenter les fichiers suivants avec de nouvelles JSP (ne pas oublier la configuration dans web.xml) :
 - delegation.jsp
 - `<html>`
 - `<head><title>JSP avec delegation</title></head>`
 - `<body>`
 - `<jsp:forward page="parametre.jsp">`
 - `<jsp:param name="nom" value="Bob" />`
 - `</jsp:forward>`
 - `</body>`
 - `</html>`
 - parametre.jsp
 - `<html>`
 - `<head><title>Affichage delegue</title></head>`
 - `<body>`
 - `<%`
 - `String nom = request.getParameter("nom");`
 - `out.println("<p>Salut " + nom + "!</p>");`
 - `%>`
 - `</body>`
 - `</html>`

Standard Tag Library

- Les JSP fournissent une bibliothèque de balises standards répondant à des besoins de base.
- Quelques domaines :
 - Core : balises permettant des fonctionnalités de base
 - XML : balises permettant de manipuler des données XML
 - I18n : balises permettant de traiter l'internationalisation
 - Database : balises permettant d'exécuter des requêtes SQL (travail normalement dédié aux JavaBeans)
- Il est possible d'étendre le langage en créant d'autres balises

Directive taglib

- Principe :
 - Inclusion d'une librairie personnalisée
- Syntaxe :
 - `<%@ taglib directive %>`
- Exemple :
 - `<%@ taglib uri="maTagLib.tld" prefix="tagExemple" %>`

Action standard : useBean

- Syntaxe :
 - `<jsp:useBean id="nom_bean" class="nom_classe" scope="type_porte" />`
- Équivalent à :
 - `<%`
 - `nom_classe obj = (nom_classe) request.getAttribute( "nom_bean" );`
 - `if ( obj== null ){`
 - `obj= new nom_classe();`
 - `type_porte.setAttribute( "nom_obj", obj);`
 - `}`
 - `%>`

Action standard : getProperty

- Obtenir la valeur d'une des propriétés d'un bean utilisé dans une JSP
- Syntaxe :
 - `<jsp:getProperty name="nom_obj_bean" property="nom_propriété" />`
- Équivalent à :
 - `<%= nom_obj_bean.getNom_propriété() %>`

Action standard : setProperty

- Modifier la valeur d'une des propriétés d'un bean utilisé dans une JSP
- Syntaxe :
 - `<jsp:setProperty name="nom_obj_bean" property="nom_propriété" value="nouvelle_valeur" />`
- Équivalent à :
 - `<%= nom_obj_bean.setNom_propriété ("nouvelle_valeur") ;%>`

Action standard : setProperty (2)

- Cette action peut prendre comme nouvelle valeur de la propriété la valeur d'un paramètre
- Syntaxe :
 - `<jsp:setProperty name="nom_obj_bean" property="nom_propriété" param="nom_parametre" />`
- Équivalent à :
 - `<%= nom_obj_bean.setNom_propriété(request.getParameter("nom_parametre")); %>`

Action standard : setProperty (3)

- Cette action peut prendre comme nouvelle valeur de la propriété la valeur d'un paramètre qui a le même identifiant que la propriété
- Syntaxe :
 - `<jsp:setProperty name="nom_obj_bean" property="nom_propriété" />`
- Équivalent à :
 - `<%=
nom_obj_bean.setNom_propriété
(request.getParameter("nom_propriété")); %>`

Action standard : setProperty (4)

- Cette action peut donner de nouvelles valeurs à plusieurs propriétés les valeurs de paramètres qui ont les mêmes identifiants que ces propriétés
- Syntaxe :
 - `<jsp:setProperty name="nom_obj_bean" property="*" />`
- Équivalent à :
 - `<%= nom_obj_bean.setNom_propriété1 (request.getParameter1("nom_propriété1")) ;%>`
 - `<%= nom_obj_bean.setNom_propriété2 (request.getParameter2("nom_propriété2")) ;%>`
 - ...

EL: Expression Language

- Spécification de EL sous l'autorité du groupe d'expert JSR-152 pour JSP 1.3
- Le JSR-52 et JSR-152 travail ensemble sur la spécification de l'Expression Language
- Deux version: une pour JSTL et l'autre pour JSP 1.3
- En JSTL il est utilisé uniquement dans la valeur d'un attribut:

```
<prefix:tag attr1="${ expr }" />
```
- Il est invoqué exclusivement via la syntaxe `${ expr }`

Expression Language EL

- **Principe :**
 - Effectuer des tests basiques sur des expressions
 - Manipuler des objets sans avoir recours au code Java
- **Avantage :**
 - + simple
 - + standard
 - + maintenable
- **Syntaxe de base :**
 - `${ expression }`
- **Exemple :**

```
<jsp:setProperty name="box" property="perimeter"  
                 value="${2*box.width+2*box.height}"/>
```

EL : Opérateurs

- Opérateurs :
 - opérateurs arithmétiques :
 - + - * / %
 - opérateurs logiques :
 - && || !
 - opérateurs relationnels :
 - == != < > <= >=
 - eq ne lt gt le ge
 - L'opérateur [] pour accéder aux objets de type Map, Array et List
 - param["p1"] → param.get("p1")

EL : Opérateur de test

- Conditionnel :
 - Syntaxe :
 - `${ condition ? 'traitement si condition vérifiée' : 'traitement sinon' }`
 - Exemple :
 - `${ true ? 'vrai' : 'faux' }`
- Opérateur « *empty* » :
 - Syntaxe :
 - `${ empty « element à tester » }`
 - Exemple :
 - `${ empty 'test' }`

EL : Manipulation des objets

- Instruction avec une action standard :
 - Prénom et nom : `<jsp:getProperty property="prenom" name="attributBean"/>`
- Instruction avec EL :
 - `${attributBean.prenom}`
 - `${attributBean.getPrenom()}`

EL : Manipulation des collections

- Possibilité de manipuler directement les collections de java.util, telque :
 - List, Set, Map ...
- Exemple d'utilisation d'une liste :
 - `<%/* Création d'une liste d'entiers et insertion de quatre éléments */`
 - `java.util.List<Integer> li = new java.util.ArrayList<Integer>();`
 - `li.add( "20" );%>`
 - `<!-- Les quatre syntaxes suivantes retournent le premier élément de la liste -->`
 - `${ li.get(0) }<br />`
 - `${ li[0] }<br />`
 - `${ li['0'] }<br />`
 - `${ li["0"] }<br />`

EL : Fonctions

- Possibilité d'utiliser des fonctions définies dans des taglib personnalisées

- Syntaxe :

```
${ns:func(param1, param2, ...)}
```

- Exemple :

```
${fn:length("Get my length")}
```

- Possibilité de définir une fonction :
 - Définition des fonctions identique à celle des tags personnalisés.

EL : Exemple de fonctions

- Définition :

```
public class MyFunctions {  
    public static String toCaps( String text ) {  
        return text.toUpperCase();  
    }  
}
```

- Utilisation :

- XML:

```
<uri>testFunctionsURI</uri>  
<function>  
    <name>toCaps</name>  
    <function-class>mypkg.MyFunctions</function-class>  
    <function-signature>String toCaps( java.lang.String)</function-signature>  
</function>
```

- JSP:

```
<%@ taglib prefix="myFunctionTest" uri="testFunctionsURI"%>  
${myFunctionTest.toCaps("testingCaps")}
```

EL : Objets implicites

Noms	Définition	Explication
pageScope	Map des attributs de portée page	Collection permettant d'accéder aux différents attributs d'une portée
requestScope	Map des attributs de portée requête	
sessionScope	Map des attributs de portée session	
applicationScope	Map des attributs de portée application	
param	Map des paramètres (String)	Collection permettant d'accéder aux paramètres de la requête HTTP
paramValues	Map des paramètres (String[])	
header	Map des valeurs de l'entête HTTP (String)	Collection permettant d'accéder à l'entête de la requête HTTP
headerValues	Map des valeurs de l'entête HTTP (String[])	
initParam	Map des init-params	Collection permettant d'accéder aux init-params définis dans web.xml
cookie	Map des différentes cookies	Collection permettant d'accéder aux cookies
pageContext	L'objet PageContext de la page courante	

EL : Exemple d'objet implicite

- Objets implicites des EL $\neq$ objets implicites des actions standards sauf « `pageContext` »
- « `pageContext` » a des propriétés pour accéder aux autres objets implicites JSP.
- Exemple : On peut accéder à l'objet `request`.

`${pageContext.request.queryString}`

Désactivation de l'évaluation des EL

- Désactivation de l'évaluation des EL (syntaxe non réservée avant JSP2) dans une JSP.

`<%@ page isELIgnored ="true|false" %>`

- Si "true" : EL ignorée
 - Si "false" : EL interprétée
 - La valeur par défaut dépend de la version du descripteur de déploiement de l'application web :
 - Servlet 2.3 et précédents : "true"
 - Servlet 2.4 et suivants : "false"
- Désactivation depuis le web.xml (pour tout le projet):

`<jsp-config>`

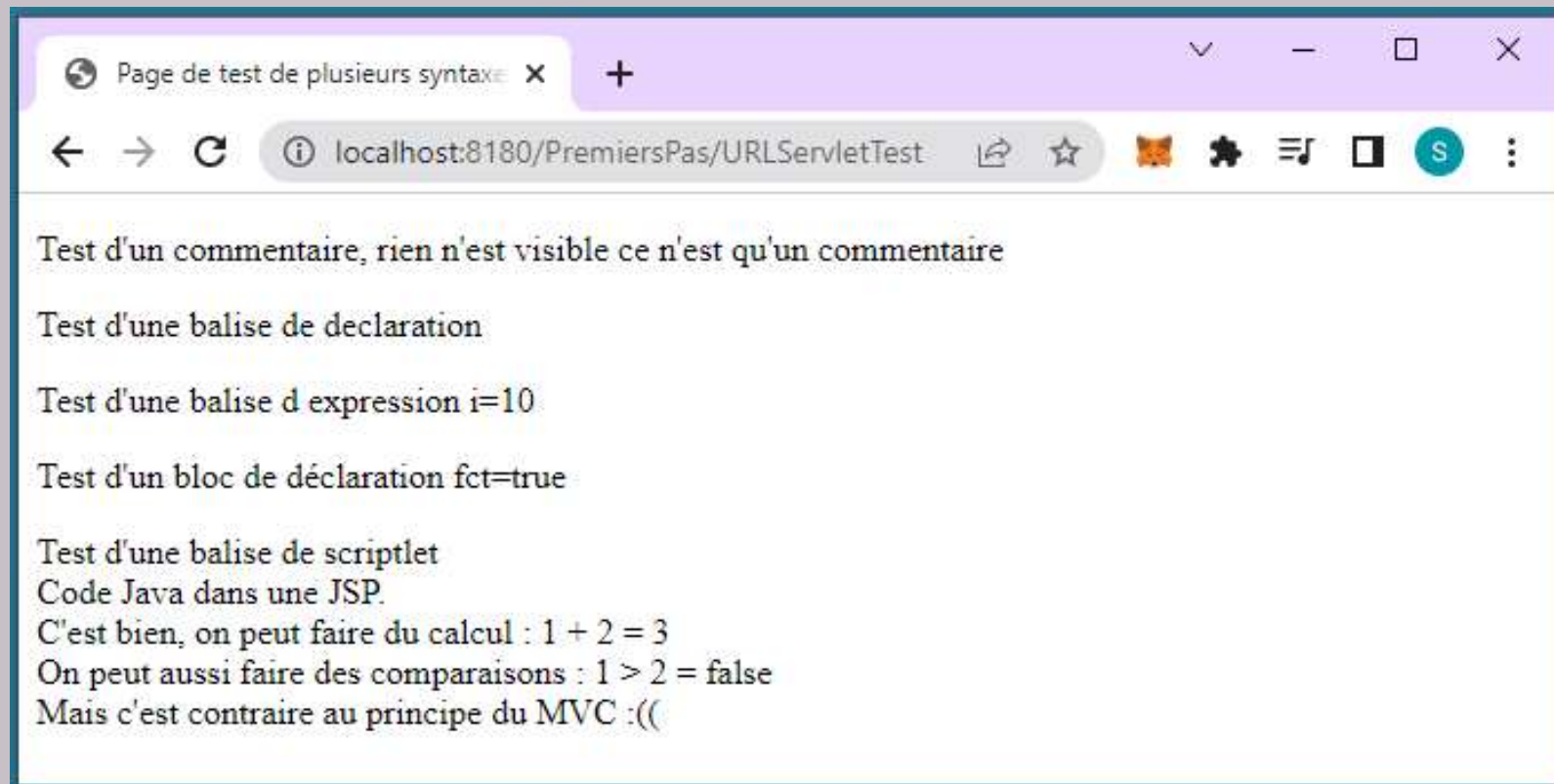
`<jsp-property-group>`

`<url-pattern>*.jsp</url-pattern>`

`<el-ignored>true</el-ignored>`

`</jsp-property-group>`

`</jsp-config>`



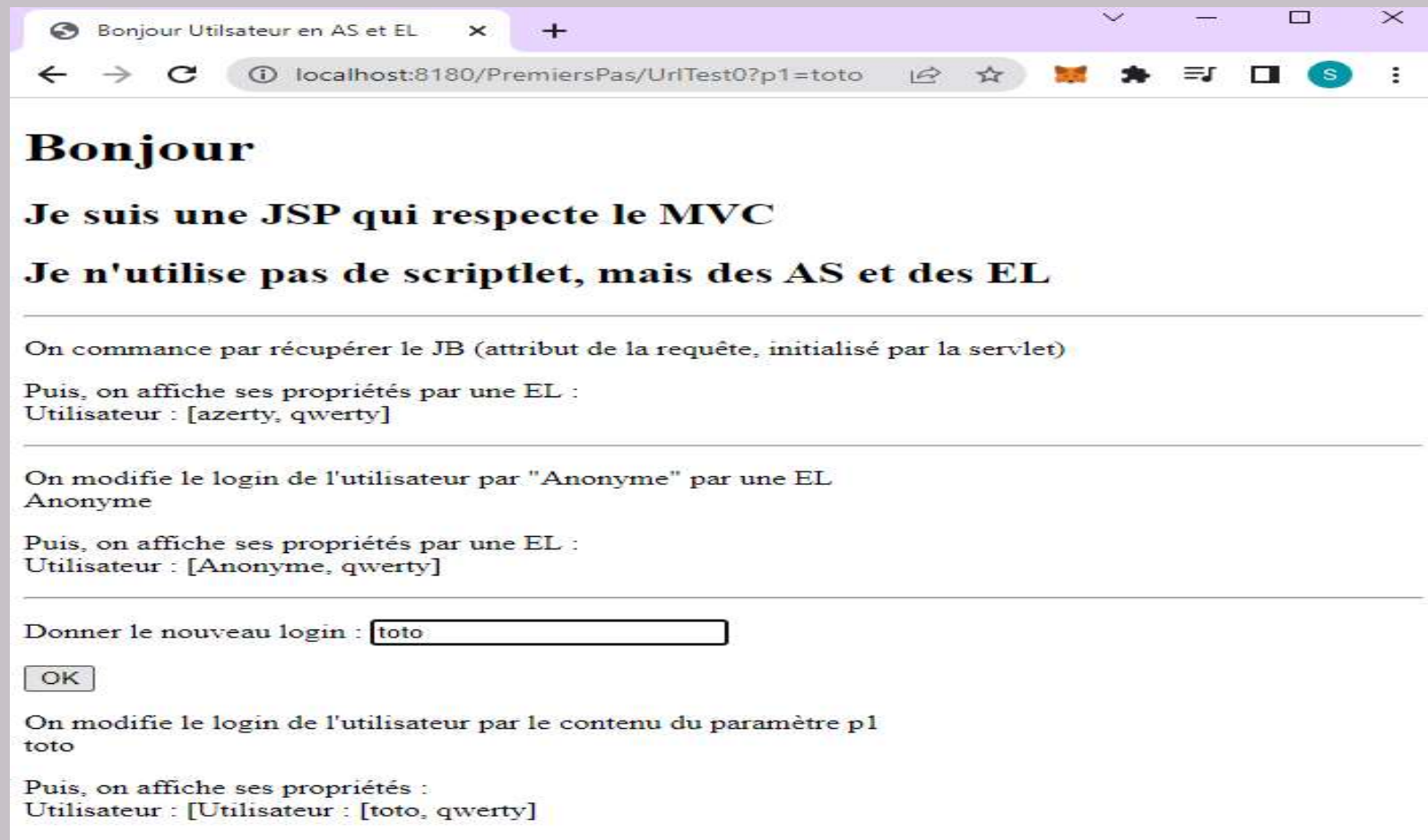
Tutoriel 5

JSP et scriptlets



Tutoriel 6

JSP et AS



Tutoriel 7

JSP et EL